Протокол виконання лабораторної роботи №2

з криптографії

Виконали студенти 3 курсу, НН ФТІ, ФБ-43

Закусило Дмитрій, Топчій Микита

Варіант 6

КРИПТОГРАФІЯ
КОМП’ЮТЕРНИЙ ПРАКТИКУМ №2

Криптоаналіз шифру Віженера

Мета роботи

Засвоєння методів частотного криптоаналізу. Здобуття навичок роботи та аналізу потокових шифрів гамування адитивного типу на прикладі шифру Віженера.

Порядок виконання роботи

1. Дослідження шифру Віженера

Самостійно підібрати текст російською мовою обсягом 2–3 КБ та підготувати його відповідно до алфавіту, прийнятого в роботі.

Реалізувати програмні засоби, необхідні для виконання завдань практикуму.

Зашифрувати один і той самий відкритий текст шифром Віженера ключами довжини r = 2, 3, 4, 5, а також одним ключем довжини 10–20 символів.

Для відкритого тексту та всіх одержаних шифртекстів обчислити індекси відповідності, порівняти їх значення та пояснити одержані результати.

Взяли частину 2-3КБ вже обробленого тексту з лаби1, перевірили аби він відповідав умовам прийнятим в лабі2.

Пояснення коду програми Task1.py та демонстрація роботи:

Серед функцій які реалізують суть цього завдання:

encrypt: приписує кожному символу ключа його індекс в алфавіті, реалізує шифр Віженера: індекс символу відкритого тексту складається з індексом відповідного символу ключа за модулем кількості символів в алфавіті. Символ ключа обирається по модулю довжини ключа. Отриманий індекс переводиться назад у символ алфавіту.

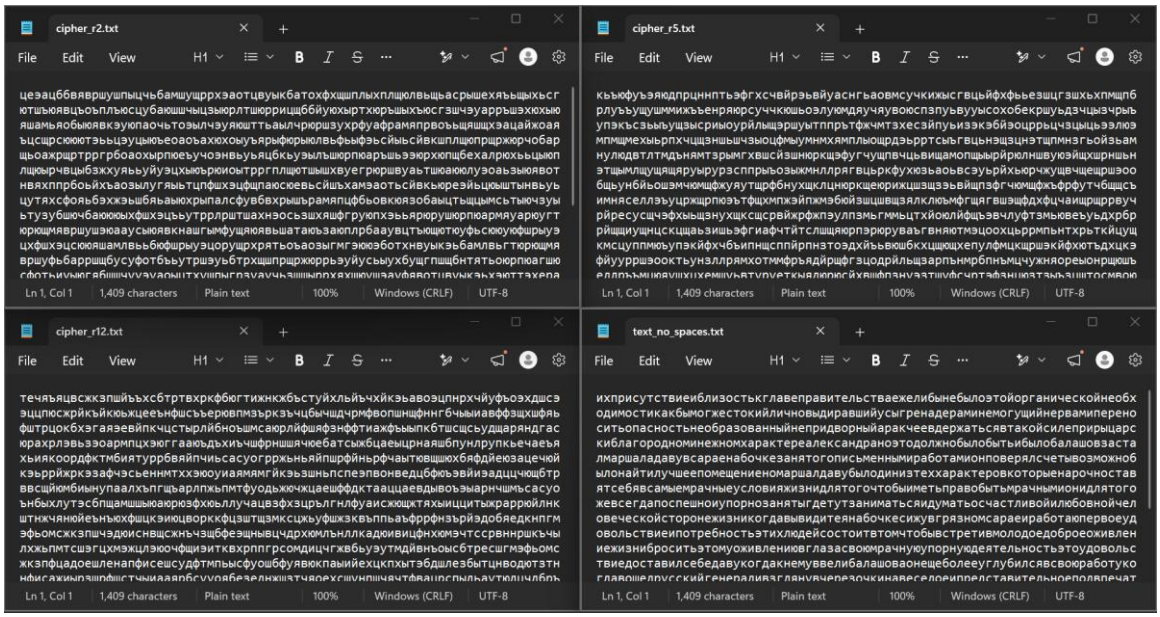
coincidence_index: рахує індекси відповідності за формулою з методички. За допомогою Counter отримуємо для кожного символа кількість його появ у

тексті. Далі застосовується формула: частка, в чисельнику сумарна кількість упорядкованих пар однакових символів, в знаменнику загальна кількість упорядкованих пар. Тобто, індекс відповідності - частка позицій, у яких дві випадково обрані букви тексту збігаються.

У функції main ці та інші функції скомпоновані разом для реалізації завдання: завантажується відкритий текст, рахується “I”, відкритий текст шифрується кожним ключем зі списку, для кожного шифротексту рахується “I”.

```
PS C:\Users\user\Desktop\Cryptography\Lab2> python -u
"c:\Users\user\Desktop\Cryptography\Lab2\Task1.py"
    plaintext: I = 0.055951
    r2: I = 0.045794
    r3: I = 0.039734
    r4: I = 0.037953
    r5: I = 0.036873
    r12: I = 0.032346
```

Запустили програму, все працює правильно: шифротексти згенерувались, а пораховані індекси є реалістичними. Для відкритого тексту значення найбільше і дорівнює теоретичному: 0.055. Зі збільшенням довжини ключа спадає індекс: імовірності символів у шифротексті дедалі більше згладжуються. 0.031 - це теоретичне значення I0=1/m=1/32 для рівномірного розподілу, r12=0.0323 наближається до цього теоретичного значення.



2. Криптоаналіз шифртексту

Для наданого викладачем шифртексту (згідно з номером варіанта) виконати криптоаналіз шифру Віженера. Довжину ключа визначати статистичними методами, описаними в розділі «Необхідні теоретичні відомості». При пошуку періоду перевірити можливі значення r щонайменше до 30. У звіті навести результати, на підставі яких було обрано довжину ключа. Недостатньо вказати лише знайдене значення r. Дослідити декілька альтернативних кандидатів на довжину ключа. Обґрунтувати вибір кандидатів та пояснити отримані результати.

Пояснення коду програми Task2.py та демонстрація роботи:

Програма завантажує шифротекст за варіантом, після чого для кожного кандидата на довжину ключа r від 2 до 35 обчислює дві статистики. Перша - усереднений індекс відповідності по блоках: шифротекст розрізається зрізами text[i::r] на r блоків (блоку r відповідають позиції i, i+r, i+2r, ...), для кожного блока викликається функція coincidence_index з task1, значення усереднюються. Друга - частка збігів D_r/(n−r): пораховується кількість позицій, у яких буква шифротексту збігається з буквою на відстані r, і нормується кількістю порівнянь, щоб статистика лежала в одній шкалі з індексом відповідності.

```
PS C:\Users\user\Desktop\Cryptography\Lab2> python -u
"c:\Users\user\Desktop\Cryptography\Lab2\Task2.py"
 r     mean I  D_r share
 2   0.034055    0.03021
 3   0.034083    0.03212
 4   0.034091    0.03752
 5   0.034123    0.03096
 6   0.033976    0.03418
 7   0.034158    0.03214
 8   0.034026    0.03302
 9   0.033919    0.03214
10   0.033929    0.03566
11   0.034009    0.03405
```

12	0.034058	0.03318
13	0.034037	0.03538
14	0.034084	0.03290
15	0.034227	0.03188
16	0.034025	0.03130
17	0.055508	0.05764
18	0.033837	0.03102
19	0.033929	0.02956
20	0.033801	0.03000
21	0.034242	0.03337
22	0.034171	0.02972
23	0.033942	0.03719
24	0.034082	0.03324
25	0.033740	0.03193
26	0.033858	0.02988
27	0.034093	0.03633
28	0.033878	0.03780
29	0.034075	0.03077
30	0.033937	0.03268
31	0.033774	0.03078
32	0.034195	0.03255
33	0.033826	0.03299
34	0.055337	0.05734
35	0.033805	0.03432

Аналіз отриманих результатів:

Для всіх перевірених значень r, крім двох, обидві статистики тримаються на фоновому рівні: середній індекс відповідності 0.033–0.034, частка збігів 0.030–0.038. Це близько до теоретичного значення для рівномірного розподілу $I_0 = 1/m = 1/32 \approx 0.031$, тобто для цих кандидатів блоки не зберігають частотних властивостей мови.

Кандидат r = 17 на їх фоні різниться: середній індекс відповідності 0.0555, частка збігів 0.0576. Обидва значення відповідають теоретичним очікуванням для тексту, що зберігає частоти російської мови (теоретичне значення≈0.05)

На кратному значенні r=34=2*17: середній індекс 0.0553, частка збігів 0.0573 - значення збігаються з показниками r=17 у межах похибки. На відстанях, кратних істинному періоду, букви шифруються однаковими зсувами, тому статистичні властивості зберігаються так само, як і при самому

періоді. Той факт, що підйом спостерігається саме при 34, а не на жодному іншому r із діапазону 2–35, узгоджується з гіпотезою r=17 і суперечить будь-якій іншій.

Тим не менше досліджено альтернативних кандидатів, які дали локальні підйоми частки збігів: r = 4 (0.0375), r = 23 (0.0372), r = 28 (0.0378). Але усі вони відхилені: середній індекс відповідності для них залишається на фоновому рівні (≈0.034), тобто підйом однієї статистики без підтвердження другою є випадковим відхиленням статистики. Якби істинним періодом був один із них, на кратних значеннях (наприклад 8, 16 для r = 4) мав би бути видимий підйом - чого не спостерігається.

3. Відновлення ключа та відкритого тексту

Після визначення довжини ключа відновити ключ за методикою, описаною в теоретичних відомостях до роботи. У звіті навести проміжні статистичні результати, достатні для обґрунтування вибору символів ключа. Розшифрувати шифртекст за знайденим ключем та перевірити отриманий результат.

Завдання виконали в два етапи: в першому ми припускали, що найпопулярніша буква кожного блоку шифротексту - «о», отримували ключ, і розшифровували виходячи з цього. Але так розшифрувати текст вийшло тільки частково: в деяких місцях припущення виявилось хибним, тому написали другу програму, яка перебирає припущення по популярності букв.

Пояснення коду програми task3_1.py та демонстрація роботи:

Нові функції, в яких зосереджена логіка програми:

caesar_shift проходить по блоку: формує список букв та кількість входжень кожної з них, топ-1 популярна буква приймається за зашифровану «о», попадає в змінну «y_star». якщо «к»-зсув(частинка ключа), «у»-шифрована буква, «х»-оригінальна буква=«о»=14, то y=x+k => k=y-x mod32. таким чином функція і шукає для кожного блоку свій «k», всього 17 зсувів, які у main зберуться в одну фразу-ключ.

decrypt - замість написання окремого циклу розшифрування, застосовуємо шифрування інвертованим ключем.

Математично (y-k) modm = (y+(-k)) modm.

main – виконує саме завдання: нарізає шифротекст на 17 блоків з кроком, рівним довжині ключа. Для кожного блока рахує свій зсув і дешифрує весь текст, виводить частину відкритого тексту, а також топ-3 букви шифротексту по популярності для кожного з 17 блоків.

```
PS C:\Users\user\Desktop\Cryptography\Lab2> python -u
"c:\Users\user\Desktop\Cryptography\Lab2\Task3_1.py"
key: возвращениеджлнда
```

head:
дорофейльвовифпсворыкобылынирэзьвжизнинепокидаизомлихотяпрожилуг
екольшешестидесяпифетработалпрораюохстроительнойкойпйниидомостройв
хэрековестолицевкрэицылюбилпорыбачипьъдрузьяминаозерэхщоганьскогокр
аядааертойгородавырэщсвалнадачномучаотуеовощиифруктывлсшитывалвнук
овавлтьезжатьзапределшрчднойвкраинын
block 0: р:47, к:33, в:32
block 1: ь:52, ц:35, о:35
block 2: х:49, м:29, й:29
block 3: р:45, к:35, з:33
block 4: ю:50, э:40, ш:32
block 5: о:50, е:33, а:31
block 6: з:41, ю:35, щ:34
block 7: у:49, е:34, к:31
block 8: ы:49, т:35, х:31
block 9: ц:57, и:31, р:29
block 10: у:40, н:34, к:27
block 11: т:54, д:38, х:28
block 12: ф:55, ж:32, о:31
block 13: щ:40, ц:40, н:29
block 14: ы:47, н:38, т:37
block 15: т:39, н:37, х:33
block 16: о:49, е:32, и:31

Аналіз отриманих результатів:

Вже на цьому етапі отримали частину ключа «возвращение». Частина тексту вже є читабельною, хоча й не повністю: «...жизнинепокида...»

Але друга половина ключа, а з нею і періодично текст відображаються некоректно. Причиною є неправильне припущення в конкретному блоці: там де незмістовна частина, найпопулярнішою буквою була не «о», як ми думали, а якась інша, тому на неправильному припущенні отримали неправильний результат, хоча в деяких місцях ми припустили правильно. Для врахування цього напишемо другу версію програми.

Таблиця частот блоків дозволяє оцінити надійність гіпотези для кожного блока окремо: у блоках, де лідер частот має значний відрив, припущення «о» спрацьовує. Таблиця вказує на малонадійність гіпотези у блоках 12–15.

Пояснення коду програми Task3_2.py та демонстрація роботи:

letter_frequencies формує частотний профіль російської мови за результатами першої лаби: читає зразок тексту лаби 1 , замінює «ё» на «е», після чого підрахунок частот виконує функція char_frequencies з того ж модуля лаби 1; результат перетворюється у словник «буква-частота».

shift_scores - для одного блока обчислює 32 оцінки, по одній на кожного кандидата зсуву k. Для кожної букви мови t береться її частота і кількість входжень у блоці її шифр-образу (t+k): сума добутків максимізується на правильному зсуві, бо при ньому вся гістограма блока накладається на частотний профіль мови. Функція Counter будує гістограму блока.

main - нарізає шифротекст на 17 блоків зрізами з кроком r; для кожного блока отримує оцінки кандидатів, сортує їх через sorted за lambda k: scores[k] і бере переможця як символ ключа. Виконує друк top-3 оцінок - проміжні статистичні результати вибору. Зібраний ключ застосовується до розшифрування: decrypt реалізує віднімання зсуву через додавання інвертованого ключа, (y−k)mod m = (y+(−k))mod m, повторно використовуючи encrypt з Task1. Розшифрований текст зберігається у decrypted.txt.

PS C:\Users\user\Desktop\Cryptography\Lab2> python -u "c:\Users\user\Desktop\Cryptography\Lab2\task3_2.py"
frequencies from ../Lab1/text_no_spaces.txt (via Lab1/task2.py):
а: 0.08233
б: 0.01778
в: 0.04534
г: 0.01969
д: 0.03038
е: 0.08241
ж: 0.01041
з: 0.01700
и: 0.06630
й: 0.01159
к: 0.03458
л: 0.05073
м: 0.02988
н: 0.06570
о: 0.11439

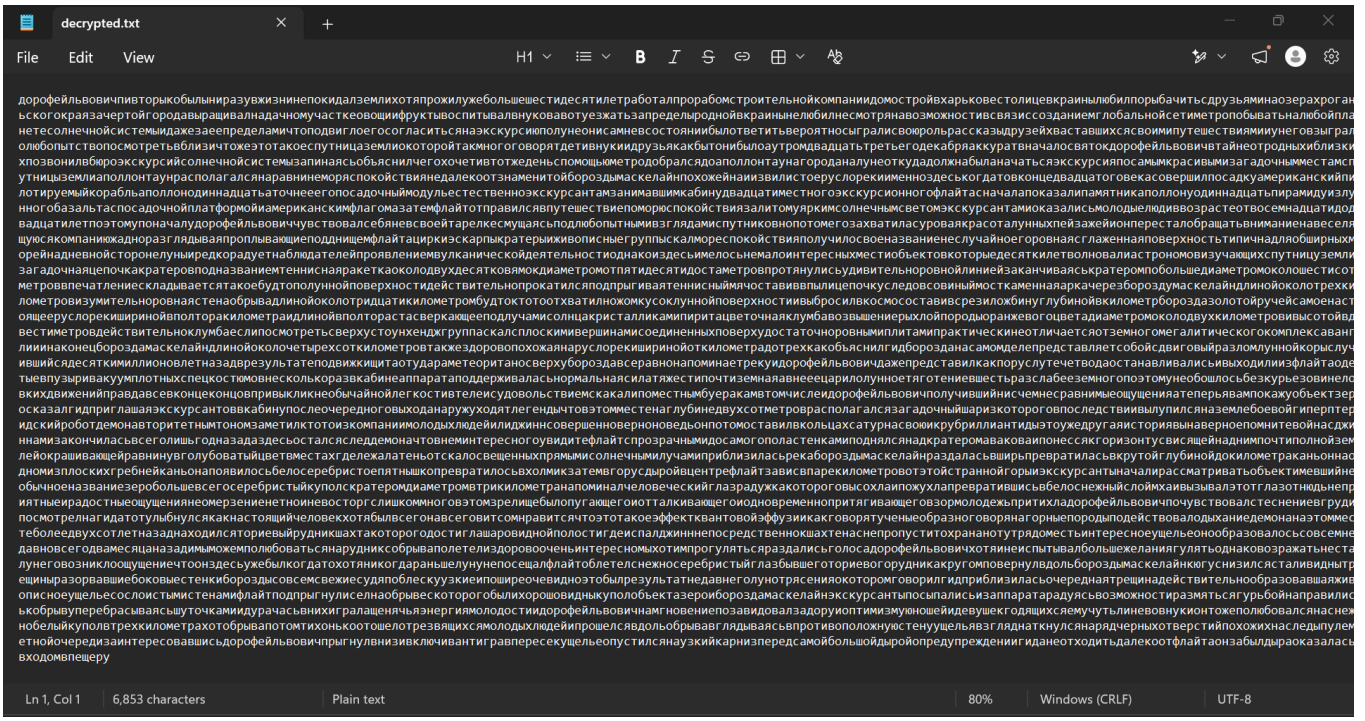
п: 0.02596
р: 0.04448
с: 0.05259
т: 0.05814
у: 0.02781
ф: 0.00197
х: 0.00867
ц: 0.00358
ч: 0.01457
ш: 0.00924
щ: 0.00284
ъ: 0.00044
ы: 0.01932
ь: 0.01986
э: 0.00320
ю: 0.00637
я: 0.02244
block 0: в:23, е:17, я:17
block 1: о:23, с:17, л:16
block 2: з:22, д:17, к:15
block 3: в:22, я:16, е:16
block 4: р:23, у:18, н:17
block 5: а:22, э:16, г:16
block 6: щ:22, ц:16, ю:15
block 7: е:22, в:17, и:16
block 8: н:22, р:17, к:16
block 9: и:22, л:16, е:16
block 10: е:22, в:17, и:17
block 11: д:23, з:16, б:16
block 12: ж:23, й:17, г:16
block 13: и:21, л:18, е:15
block 14: н:23, к:16, р:15
block 15: н:22, к:16, р:16
block 16: а:23, г:17, э:16
key: возвращениеджинна
head:
дорофейльвовичпивторыкобылыниразувжизнинепокидалземлихотяпрожилужебольшешестидесятилетработалпрорабомстроительнойкомпаниидомостройвхарьковестолицевкраинылюбилпорыбачитьсдрузьяминаозерахроганьскогокразачертойгородавыращивалнадачномучасткеовощиифруктывоспитывалвнуковавотуезжатьзапределыроднойвкраинын

Аналіз отриманих результатів:

Частоти, отримані з лаби 1, узгоджується з очікуваннями. Найбільша частота «о» обґрунтовує вибране припущення для першої версії програми і водночас показує недолік цієї однієї гіпотези: на блоках по 400 символів частоти «о» та «е» близькі, тому на окремому блоці топ-1 може стати «е» або інша буква - що і сталося у блоках 11–16 при запуску першої програми.

Для кожного з 17 блоків обрано зсув з максимальною оцінкою; відрив першого кандидата присутній з запасом у кожному блоці. Отриманий ключ: «возвращениеджинна» - осмислена фраза. Порівняно з першим прогоном: символи блоків 0–12,14,16 збіглися, а пара виправлених символів «джинна» є результатом корекції: заміна одної гіпотези гіпотези «топ-буква = о» на порівняння всіх 32 зсувів за розподілом частот.

Відкритий текст читається зв’язно. У першій програмі саме на позиціях неправильно визначених блоків знаходилися неправильно розшифровані частини тексту.



4. Дослідження впливу помилки у ключі

Змінити один довільний символ правильно відновленого ключа та повторно розшифрувати той самий шифртекст. До проведення експерименту спрогнозувати, як така зміна вплине на результат розшифрування. Порівняти прогноз із результатом експерименту та пояснити отриманий ефект.

Прогноз: після зміни одного символа ключа неправильними стануть лише ті позиції тексту, що шифрувались цим символом ключа - кожна 17-та буква, починаючи з позиції, рівної номеру зміненого символа, причому кожна помилкова буква буде зміщена на одну й ту саму величину дельта=новий зсув – старий зсув mod32. Решта тексту залишиться правильною.

Пояснення коду програми Task4.py та демонстрація роботи:

Програма використовує encrypt з Task1, decrypt реалізує розшифрування через додавання інвертованого ключа, (y − k) mod m = (y + (−k)) mod m.

У main спочатку формується спотворений ключ: з ключа «возвращениеджинна» вирізанням та підстановкою замінюється один символ на позиції 5 - «а» на «ф». Далі обчислюється очікуваний зсув помилок дельта = (номер нової букви - номер старої) mod 32. Текст розшифровується двічі - правильним та спотвореним ключем - і результати порівнюються посимвольно: у список errors потрапляють позиції розбіжностей. Для перевірки прогнозу виводиться множина залишків цих позицій від ділення на 17, а також перші 15 пар «правильна буква - помилкова буква» та початок тексту, розшифрованого зі спотвореним ключем.

```
PS C:\Users\user\Desktop\Cryptography\Lab2> python -u
"c:\Users\user\Desktop\Cryptography\Lab2\Task4.py"
original key: возвращениеджинна
modified key: возврфщениеджинна
delta = 20 (ф)
total mismatches: 403
error positions mod 17: [5]
pos 5: е become с
pos 22: о become ъ
```

pos 39: н become щ
pos 56: я become л
pos 73: е become с
pos 90: т become ю
pos 107: е become с
pos 124: о become ъ
pos 141: о become ъ
pos 158: п become ы
pos 175: м become ш
pos 192: к become ц
pos 209: о become ъ
pos 226: а become м
pos 243: и become ф
head with wrong key:
дорофсйльвовичпивторыкъбылыниразувжизнищепокидалземлихотлпрожилужебольшесстидесятилетрабоюалпрорабомстроитсльнойкомпани

Аналіз отриманих результатів:

Прогноз виправдався: 6853символи тексту, 6853/17≈403 помилки. Множина залишків на місцях помилок=5, адже неправильний символ ключа знаходиться саме на цьому індексі, і не розповсюджується на інші позиції тексту. Кожна помилкова буква дорівнює правильній, зміщеній на −20 mod 32 (е-с, о-ъ, н-щ тощо), що видно з наведених пар. Це є наслідки структури шифру - кожен символ шифрується незалежно. Але навіть так, текст все ще є здебільшого читабельним.

Питання для аналізу результатів

1. Як змінювався індекс відповідності шифртексту при використанні ключів різної довжини? Поясніть отриманий результат.

Індекс відповідності спадав зі збільшенням довжини ключа: 0.0458 (r=2), 0.0397 (r=3), 0.0380 (r=4), 0.0369 (r=5), 0.0323 (r=12) при 0.0559 у відкритому тексті. Кожен блок шифрується шифром Цезаря і зберігає нерівномірність частот мови, але з ростом r блоки коротшають, статистика на них стає меншою, і розподіл шифртексту наближається до рівномірного I0= 1/32 ≈ 0.031).

2. За якими ознаками було визначено найбільш імовірну довжину невідомого ключа?

За двома незалежними статистиками при переборі r = 2..35: усередненим по блоках індексом відповідності та часткою збігів D_r/(n−r). Кандидат r = 17 дав 0.0555 та 0.0576 відповідно при ≈0.033-0.034 у решті, а кратне значення r = 34 повторило підйом (0.0553, 0.0573), що підтвердило період.

3. Які альтернативні кандидати на довжину ключа ви досліджували, чому обрали саме їх та які результати отримали?

Альтернативними кандидатами розглядали r = 4, 23, 28 - значення з локальними піками частки збігів (0.0375, 0.0372, 0.0378). Усі відхилені: середній індекс відповідності для них лишався на загальному рівні (≈0.034), тобто підйом однієї статистики без підтвердження другою є випадковим відхиленням. До того ж на кратних їм значеннях підйому немає.

4. Яка статистична інформація використовувалася для відновлення окремих символів ключа?

Частоти букв мови з лаби1 та гістограми частот кожного блоку шифротексту: для кожного блока перебирали всі 32 зсуви й обирали той, за якого сума «частота букви мови * кількість її шифр-образу в блоці» максимальна.

5. Як можна перевірити правильність відновленого ключа, не знаючи початкового відкритого тексту?

За осмисленістю розшифрованого тексту і, в межах даної лаби, осмисленістю самого ключа. Статистично правильність можна перевірити повторним обчисленням індексу відповідності розшифрованого тексту: він має бути на рівні мови (≈0.05).

6. Як змінився результат розшифрування після внесення помилки в один символ ключа? Поясніть характер і періодичність змін.

Спотворились лише позиції, що шифрувались зміненим символом ключа: 403 з 6853 символів, усі з залишком 5 mod 17 - періодичність рівна довжині ключа. Кожна помилкова буква зміщена на одну й ту саму величину (−20 mod 32) відносно правильної, а решта тексту не змінилась, бо шифр обробляє кожен символ незалежно і помилка не розповсюджується.

7. Які етапи проведеного криптоаналізу найбільше залежать від обсягу доступного шифртексту? Чому?

Знаходження періоду та відновлення символів ключа: обидві статистики періоду (індекс відповідності блоків, D_r) ґрунтуються на частотних оцінках, точність яких зростає з обсягом. Аналогічно надійність вибору зсуву блока залежить від кількості букв у ньому.

8. Які результати ваших експериментів найкраще демонструють слабкість шифру Віженера з періодичним ключем?

Перебір r = 2..35: уся довжина ключа визначається з шифротексту без жодного знання ключа, після чого ключ відновлюється по блоках звичайним частотним аналізом шифру Цезаря. Також сюди ж можна назвати повне відновлення осмисленого 17-символьного ключа і тексту з 6800 символів: періодичність ключа робить поліалфавітний шифр набором незалежних моноалфавітних, які ламаються по одному.

Висновки

У ході роботи ми засвоїли методи частотного криптоаналізу та набули навичок аналізу потокових шифрів на прикладі шифру Віженера. Дослідили залежність індексу відповідності від довжини ключа: з ростом r розподіл шифртексту наближається до рівномірного. Реалізували два незалежні методи визначення періоду і навчились узгоджувати їхні показання. Зрозуміли механізм відновлення ключа по блоках шифру Цезарях: дійшли до порівняння всіх 32 зсувів за повним частотним профілем мови, що дозволило виправити помилкові символи. Дослідили вплив помилки в ключі: спотворення локальні, періодичні з періодом ключа й однакові за величиною зсуву. Загалом зрозуміли, що періодичний ключ робить шифр Віженера вразливим до статистичного криптоаналізу.